

**UAS**  
**PENGOLAHAN CITRA**



NAMA : Dean Hukama Wirasena

NIM : 202431071

KELAS : C

DOSEN : Darma Rusjdi., Ir., M.Kom

NO.PC : 18

ASISTEN : 1. Muhammad Farhan Fahrezy

2. Rizqy Amanda

**INSTITUT TEKNOLOGI PLN**  
**TEKNIK INFORMATIKA**  
**2025/2026**

## DAFTAR ISI

|                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------|----|
| DAFTAR ISI.....                                                                                              | 2  |
| BAB I.....                                                                                                   | 4  |
| PENDAHULUAN .....                                                                                            | 4  |
| <b>1.1 Latar Belakang</b> .....                                                                              | 4  |
| <b>1.2 Rumusan Masalah</b> .....                                                                             | 4  |
| <b>1.3 Tujuan</b> .....                                                                                      | 4  |
| <b>1.4 Manfaat</b> .....                                                                                     | 4  |
| BAB II.....                                                                                                  | 5  |
| LANDASAN TEORI.....                                                                                          | 5  |
| <b>2.1 Citra digital dan Filtering Citra</b> .....                                                           | 5  |
| <b>2.2 Jenis-Jenis Noise pada Citra Digital</b> .....                                                        | 6  |
| <b>2.3 Filtering pada Domain Spasial dan Domain Frekuensi</b> .....                                          | 6  |
| <b>2.3.1 Filtering Domain Spasial</b> .....                                                                  | 6  |
| <b>2.4 Perbedaan filter Linear dan filter Non Linear</b> .....                                               | 7  |
| <b>2.4.1 Filter Linear</b> .....                                                                             | 7  |
| <b>2.4.2 Filter Nonlinear</b> .....                                                                          | 7  |
| <b>2.5 Karakteristik Mean Filter, Gaussian Filter, Median Filter, Bilateral Filter, dan Filter Penajaman</b> | 8  |
| <b>2.5.1 Mean Filter</b> .....                                                                               | 8  |
| <b>2.5.2 Gaussian Filter</b> .....                                                                           | 8  |
| <b>2.5.3 Median Filter</b> .....                                                                             | 9  |
| <b>2.5.4 Bilateral Filter</b> .....                                                                          | 9  |
| <b>2.5.5 Filter Penajaman</b> .....                                                                          | 9  |
| BAB III .....                                                                                                | 10 |
| HASIL .....                                                                                                  | 10 |
| <b>3.1 Gambaran Umum</b> .....                                                                               | 10 |
| <b>3.2 Perangkat yang digunakan</b> .....                                                                    | 10 |
| <b>3.3 Input Citra</b> .....                                                                                 | 11 |
| <b>3.4 Penampakan Citra Asli</b> .....                                                                       | 12 |
| <b>3.5 Median Filtering</b> .....                                                                            | 12 |
| <b>3.6 Konversi Grayscale</b> .....                                                                          | 13 |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>3.7 Mean Filtering Manual .....</b>        | <b>14</b> |
| <b>3.8 Perbandingan Hasil Filtering .....</b> | <b>15</b> |
| <b>3.9 Penyimpanan Hasil Citra.....</b>       | <b>15</b> |
| <b>BAB IV .....</b>                           | <b>17</b> |
| <b>PENUTUP.....</b>                           | <b>17</b> |
| <b>4.1 Kesimpulan .....</b>                   | <b>17</b> |
| <b>4.2 Saran.....</b>                         | <b>18</b> |
| <b>DAFTAR PUSTAKA .....</b>                   | <b>20</b> |

## **BAB I**

### **PENDAHULUAN**

#### **1.1 Latar Belakang**

Perkembangan teknologi digital menyebabkan citra menjadi salah satu bentuk informasi yang paling banyak digunakan. Citra digital dapat diperoleh melalui kamera, pemindai, satelit, mikroskop, perangkat medis, sensor industri, dan berbagai perangkat berbasis visi komputer. Penggunaannya mencakup bidang fotografi, keamanan, kesehatan, penginderaan jauh, robotika, kendaraan otonom, identifikasi wajah, inspeksi produk, hingga dokumentasi ilmiah.

Noise pada citra dapat muncul akibat keterbatasan proses akuisisi, sensor, penyimpanan, dan transmisi sehingga diperlukan metode denoising untuk memperbaiki kualitas citra (Elad et al., 2023). Dalam proses pembentukan citra digital, kualitas citra yang dihasilkan tidak selalu sempurna. Citra dapat mengalami gangguan berupa noise, perubahan pencahayaan, kekaburan, distorsi, atau hilangnya sebagian informasi. Noise dapat muncul ketika citra ditangkap oleh sensor, disimpan, dikompresi, maupun dikirimkan melalui saluran komunikasi. Faktor lingkungan seperti pencahayaan yang rendah dan suhu sensor juga dapat memengaruhi kualitas citra. Penelitian terkini menjelaskan bahwa noise umumnya berasal dari proses akuisisi dan transmisi citra, termasuk keterbatasan sensor serta ketidaksempurnaan perangkat pengirim, penerima, dan saluran komunikasi.

#### **1.2 Rumusan Masalah**

1. Apa yang dimaksud dengan citra digital dan filtering citra?
2. Apa saja jenis noise yang dapat ditemukan pada citra digital?
3. Bagaimana prinsip kerja filtering pada domain spasial?
4. Apa perbedaan filter linear dan filter nonlinier?
5. Bagaimana karakteristik mean filter, Gaussian filter, median filter, bilateral filter, dan filter penajaman?

#### **1.3 Tujuan**

1. Menjelaskan konsep dasar citra digital dan filtering citra.
2. Menjelaskan jenis-jenis noise pada citra digital.
3. Menguraikan filtering pada domain spasial dan domain frekuensi.
4. Menjelaskan prinsip kerja berbagai jenis filter.
5. Membandingkan kelebihan serta kekurangan setiap filter.

#### **1.4 Manfaat**

1. Menambah pemahaman mengenai konsep dan jenis-jenis filtering citra.
2. Membantu memilih metode filter yang sesuai untuk mengurangi noise.
3. Membantu mempertahankan detail dan meningkatkan kualitas citra.

## BAB II

### LANDASAN TEORI

#### 2.1 Citra digital dan Filtering Citra

Citra digital merupakan representasi suatu objek atau keadaan dalam bentuk data numerik yang dapat disimpan dan diolah menggunakan komputer. Citra digital tersusun atas sejumlah elemen kecil yang disebut piksel atau *picture element*. Setiap piksel memiliki nilai tertentu yang menunjukkan tingkat intensitas cahaya atau warna pada posisi tersebut.

Secara matematis, citra digital dapat dinyatakan sebagai fungsi dua dimensi:

$$f(x, y)$$

Keterangan:

- $x$  menunjukkan posisi piksel secara horizontal.
- $y$  menunjukkan posisi piksel secara vertikal.
- $f(x, y)$  menunjukkan nilai intensitas piksel pada koordinat tersebut.

Pada citra grayscale, setiap piksel hanya memiliki satu nilai intensitas. Citra grayscale 8-bit umumnya memiliki rentang nilai 0 sampai 255. Nilai 0 menunjukkan warna hitam, nilai 255 menunjukkan warna putih, sedangkan nilai di antara keduanya menunjukkan berbagai tingkat keabuan. Berbeda dengan citra grayscale, citra berwarna umumnya menggunakan model warna RGB atau Red, Green, Blue. Setiap piksel memiliki tiga komponen warna, yaitu merah, hijau, dan biru. Kombinasi nilai ketiga komponen tersebut menghasilkan berbagai warna yang dapat dilihat pada citra.

Salah satu proses penting dalam pengolahan citra digital adalah filtering citra. Filtering citra mengubah nilai piksel untuk menghasilkan penghalusan, pengurangan noise, atau penonjolan karakteristik tertentu pada citra (Desai et al., 2022). Proses tersebut dilakukan untuk mengurangi noise, memperhalus citra, mempertajam detail, atau menonjolkan bagian tertentu. Filtering umumnya menggunakan matriks kecil yang disebut kernel, mask, atau filter window. Kernel digeser ke seluruh bagian citra. Pada setiap posisi, nilai piksel keluaran dihitung berdasarkan nilai piksel yang berada di dalam daerah kernel.

Contoh kernel sederhana berukuran  $3 \times 3$  adalah:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Ukuran kernel yang sering digunakan adalah  $3 \times 3$ ,  $5 \times 5$ , dan  $7 \times 7$ . Kernel yang lebih besar dapat memberikan efek filtering yang lebih kuat. Namun, penggunaan kernel besar juga dapat menghilangkan detail penting serta memperbesar waktu komputasi.

Secara umum, filtering citra memiliki beberapa tujuan. Pertama, mengurangi noise yang mengganggu tampilan citra. Kedua, memperhalus perbedaan intensitas yang terlalu tajam. Ketiga, memperjelas tepi dan detail objek. Keempat, mempersiapkan citra untuk proses lanjutan seperti segmentasi, pengenalan pola, dan klasifikasi. Hasil filtering harus disesuaikan dengan kebutuhan. Filtering yang terlalu lemah menyebabkan noise masih terlihat, sedangkan filtering yang terlalu kuat dapat menghilangkan detail. Oleh karena itu, pemilihan metode dan ukuran kernel perlu mempertimbangkan karakteristik citra serta jenis gangguan yang terdapat di dalamnya.

## 2.2 Jenis-Jenis Noise pada Citra Digital

Noise merupakan gangguan yang menyebabkan nilai piksel pada citra berbeda dari nilai yang seharusnya. Noise dapat muncul dalam bentuk titik-titik acak, bintik hitam dan putih, pola kasar, atau perubahan intensitas yang tidak beraturan.

Secara sederhana, citra yang mengandung noise dapat dituliskan sebagai:

$$g(x,y)=f(x,y)+n(x,y)$$

Keterangan:

- $g(x,y)$  merupakan citra yang mengandung noise.
- $f(x,y)$  merupakan citra asli.
- $n(x,y)$  merupakan komponen noise.

Noise dapat muncul ketika citra diambil menggunakan kamera, dipindai, disimpan, dikompresi, atau dikirim melalui jaringan. Kondisi pencahayaan yang buruk, kualitas sensor yang rendah, suhu perangkat, serta gangguan pada media komunikasi juga dapat menghasilkan noise. Keberadaan noise dapat mengurangi kualitas visual citra dan mengganggu proses analisis. Pada proses deteksi tepi, noise dapat dianggap sebagai tepi palsu. Pada proses segmentasi, noise dapat menyebabkan objek terpecah menjadi bagian-bagian kecil. Pada proses klasifikasi, noise dapat mengubah karakteristik citra sehingga menurunkan tingkat akurasi.

## 2.3 Filtering pada Domain Spasial dan Domain Frekuensi

Filtering citra dapat dilakukan melalui dua pendekatan utama, yaitu filtering pada domain spasial dan filtering pada domain frekuensi. Kedua pendekatan tersebut memiliki tujuan yang sama, tetapi menggunakan cara kerja yang berbeda.

### 2.3.1 Filtering Domain Spasial

Domain spasial merupakan domain yang berhubungan langsung dengan posisi dan nilai piksel pada citra. Filtering domain spasial dilakukan dengan mengubah nilai piksel secara langsung berdasarkan nilai piksel-piksel yang berada di sekitarnya.

Secara umum, filtering domain spasial dapat dituliskan sebagai:

$$g(x,y)=T[f(x,y)]$$

Keterangan:

- $f(x,y)$  merupakan citra masukan.
- $T$  merupakan operasi filtering.
- $g(x,y)$  merupakan citra keluaran.

Pada filtering spasial, kernel digeser dari satu piksel ke piksel lainnya. Setiap nilai piksel keluaran dihitung menggunakan operasi tertentu, misalnya penjumlahan, rata-rata, nilai median, nilai maksimum, atau nilai minimum. Nilai piksel hasil diperoleh dengan menjumlahkan sembilan piksel yang berada di dalam kernel, kemudian membaginya dengan sembilan. Filtering domain spasial dapat dibagi menjadi dua kelompok berdasarkan tujuannya, yaitu smoothing dan sharpening. Smoothing digunakan untuk memperhalus citra dan mengurangi noise. Sharpening digunakan untuk mempertegas tepi dan detail citra.

Keunggulan filtering domain spasial adalah cara kerjanya mudah dipahami dan diimplementasikan. Filter dapat langsung diterapkan pada nilai piksel tanpa harus melakukan transformasi terlebih dahulu. Kekurangannya adalah proses filtering dapat membutuhkan waktu lama apabila ukuran citra dan kernel sangat besar. Selain itu, filter spasial tertentu dapat menyebabkan detail dan tepi objek menjadi kabur.

## **2.4 Perbedaan filter Linear dan filter Non Linear**

### **2.4.1 Filter Linear**

Filter linear merupakan filter yang menghitung nilai piksel keluaran sebagai kombinasi linear dari piksel-piksel di sekitarnya. Setiap nilai piksel dikalikan dengan bobot tertentu, kemudian hasilnya dijumlahkan. Pada mean filter, seluruh piksel di dalam kernel memiliki bobot yang sama. Pada Gaussian filter, piksel yang berada dekat dengan pusat kernel memiliki bobot lebih besar dibandingkan piksel yang berada lebih jauh.

Filter linear memiliki beberapa kelebihan. Pertama, metode ini relatif sederhana dan mudah diterapkan. Kedua, perhitungannya dapat dilakukan menggunakan operasi konvolusi. Ketiga, filter linear cukup efektif untuk mengurangi Gaussian noise ringan. Kekurangan filter linear adalah kemampuannya dalam mempertahankan tepi relatif terbatas. Filter ini memperlakukan noise dan detail dengan cara yang hampir sama. Akibatnya, proses smoothing dapat menyebabkan tepi dan tekstur citra menjadi kabur. Filter linear juga sensitif terhadap nilai ekstrem. Pada mean filter, satu piksel dengan nilai yang sangat tinggi atau sangat rendah dapat memengaruhi nilai rata-rata secara signifikan.

### **2.4.2 Filter Nonlinear**

Filter nonlinier merupakan filter yang hubungan antara masukan dan keluarannya tidak dapat dinyatakan sebagai kombinasi linear sederhana. Nilai keluaran dapat ditentukan berdasarkan pengurutan, kemiripan intensitas, kondisi tertentu, atau karakteristik lokal citra. Pada median filter, nilai piksel keluaran diperoleh dengan mengurutkan nilai piksel dalam kernel, kemudian memilih nilai tengah. Pada bilateral filter, nilai keluaran ditentukan berdasarkan jarak spasial dan kemiripan intensitas.

Filter nonlinier biasanya lebih baik dalam mempertahankan tepi dibandingkan filter linear. Filter ini juga efektif untuk menangani noise yang memiliki nilai ekstrem, seperti salt-and-pepper noise. Namun, proses perhitungan filter nonlinier umumnya lebih kompleks. Median filter memerlukan proses pengurutan nilai piksel, sedangkan bilateral filter memerlukan perhitungan bobot berdasarkan dua faktor.

Perbedaan utama filter linear dan nonlinier terletak pada cara menghitung nilai piksel keluaran. Filter linear menggunakan penjumlahan berbobot, sedangkan filter nonlinier menggunakan aturan yang tidak bersifat linear. Filter linear lebih sesuai digunakan ketika kecepatan pemrosesan dan kesederhanaan menjadi prioritas. Filter nonlinier lebih sesuai ketika citra memiliki noise ekstrem atau ketika tepi objek perlu dipertahankan.

## **2.5 Karakteristik Mean Filter, Gaussian Filter, Median Filter, Bilateral Filter, dan Filter Penajaman**

### **2.5.1 Mean Filter**

Filter linear berbasis rata-rata dapat menghasilkan penghalusan citra, tetapi berisiko mengaburkan detail dan batas objek karena seluruh piksel dalam kernel berkontribusi terhadap nilai keluaran (Satpathy et al., 2022). Mean filter merupakan filter smoothing yang mengganti nilai piksel pusat dengan nilai rata-rata piksel yang berada di dalam kernel. Kernel yang digunakan biasanya berukuran  $3 \times 3$ ,  $5 \times 5$ , atau  $7 \times 7$ . Sebagai contoh, pada kernel berukuran  $3 \times 3$  terdapat sembilan nilai piksel. Seluruh nilai tersebut dijumlahkan, kemudian dibagi dengan jumlah piksel untuk memperoleh nilai rata-rata. Nilai rata-rata tersebut selanjutnya digunakan untuk menggantikan nilai piksel pusat.

Mean filter dapat mengurangi variasi intensitas dan noise ringan. Namun, karena seluruh piksel memperoleh bobot yang sama, filter ini dapat menyebabkan tepi objek menjadi lebih kabur. Kelebihan mean filter adalah proses perhitungannya sederhana dan cepat. Kekurangannya adalah sensitif terhadap nilai ekstrem dan kurang efektif untuk menangani salt-and-pepper noise.

### **2.5.2 Gaussian Filter**

Gaussian filter merupakan filter smoothing yang menggunakan bobot berbeda pada setiap piksel di dalam kernel. Piksel yang berada lebih dekat dengan pusat kernel memiliki bobot lebih besar dibandingkan piksel yang berada lebih jauh. Cara kerja tersebut membuat Gaussian filter menghasilkan penghalusan yang lebih alami dibandingkan mean filter. Nilai piksel pusat memiliki pengaruh paling besar, sedangkan piksel di bagian tepi kernel memiliki pengaruh lebih kecil.

Tingkat penghalusan Gaussian filter dipengaruhi oleh ukuran kernel dan nilai parameter sigma. Semakin besar nilai sigma, semakin kuat efek blur yang dihasilkan. Kelebihan Gaussian filter adalah efektif untuk mengurangi Gaussian noise dan menghasilkan perubahan intensitas yang halus. Kekurangannya adalah masih dapat mengaburkan tepi serta detail kecil pada citra.



### 2.5.3 Median Filter

Median filter termasuk filter nonlinier yang efektif dalam menangani nilai piksel ekstrem dan mempertahankan batas objek lebih baik daripada filter rata-rata pada kondisi tertentu (Rezaee, 2021). Cara kerjanya dilakukan dengan mengurutkan seluruh nilai piksel dari yang terkecil hingga terbesar. Setelah itu, nilai yang berada di posisi tengah dipilih sebagai nilai piksel baru. Sebagai contoh, terdapat nilai piksel 10, 11, 12, 13, 14, 15, 16, 17, dan 255. Setelah diurutkan, nilai tengahnya adalah 14. Nilai 255 yang sangat tinggi tidak memberikan pengaruh besar terhadap hasil filtering.

Median filter sangat efektif untuk mengurangi salt-and-pepper noise. Filter ini juga lebih mampu mempertahankan tepi dibandingkan mean filter. Kekurangan median filter adalah dapat menghilangkan detail kecil apabila ukuran kernel terlalu besar. Selain itu, proses pengurutan nilai membuat perhitungannya lebih berat dibandingkan mean filter.

### 2.5.4 Bilateral Filter

Bilateral filter merupakan filter nonlinier yang mempertimbangkan dua hal, yaitu jarak antar-piksel dan perbedaan nilai intensitas. Piksel yang berada dekat dengan piksel pusat akan memiliki pengaruh lebih besar. Namun, piksel yang memiliki perbedaan intensitas terlalu jauh akan memperoleh pengaruh yang lebih kecil, meskipun posisinya berdekatan.

Cara kerja tersebut membuat bilateral filter mampu mengurangi noise tanpa terlalu mengaburkan tepi objek. Piksel yang berada di sisi berbeda dari suatu batas objek biasanya memiliki perbedaan intensitas besar, sehingga tidak banyak memengaruhi hasil filtering. Kelebihan bilateral filter adalah mampu menghasilkan citra yang lebih halus sekaligus mempertahankan batas objek. Kekurangannya adalah proses perhitungannya lebih berat dan pemilihan parameter harus dilakukan dengan tepat.

### 2.5.5 Filter Penajaman

Filter penajaman digunakan untuk memperjelas detail, garis, dan batas objek pada citra. Filter ini bekerja dengan memperkuat perubahan intensitas yang terjadi secara cepat. Salah satu jenis filter penajaman adalah Laplacian filter. Filter ini menonjolkan perbedaan antara piksel pusat dan piksel-piksel di sekitarnya. Hasilnya, bagian tepi objek akan terlihat lebih jelas.

Metode penajaman lainnya adalah unsharp masking. Metode ini dilakukan dengan membuat versi citra yang telah dihaluskan, kemudian membandingkannya dengan citra asli. Perbedaan antara keduanya dianggap sebagai detail citra. Detail tersebut kemudian ditambahkan kembali ke citra asli agar hasilnya terlihat lebih tajam. Filter penajaman dapat memperjelas objek yang tampak buram. Namun, filter ini juga dapat memperkuat noise. Oleh karena itu, citra yang memiliki banyak noise sebaiknya melalui proses smoothing terlebih dahulu sebelum dilakukan penajaman.

## BAB III

### HASIL

#### 3.1 Gambaran Umum

Pada tugas ini dilakukan pengolahan citra digital menggunakan dua metode filtering, yaitu median filtering dan mean filtering. Citra yang digunakan merupakan foto diri yang diambil dari jarak minimal dua meter sesuai dengan ketentuan tugas. Foto tersebut kemudian diproses menggunakan bahasa pemrograman Python melalui Jupyter Notebook. Proses pengolahan citra menggunakan beberapa pustaka, yaitu OpenCV, NumPy, Matplotlib, dan Pathlib. OpenCV digunakan untuk membaca citra, melakukan konversi warna, menerapkan median filtering, dan menyimpan hasil citra. NumPy digunakan untuk melakukan perhitungan mean filtering secara manual. Matplotlib digunakan untuk menampilkan citra asli dan hasil filtering. Pathlib digunakan untuk memeriksa keberadaan file citra sebelum program membacanya.

Tahapan pengolahan dimulai dari instalasi pustaka, pembacaan citra, konversi warna, penerapan median filtering, konversi grayscale, perhitungan mean filtering secara manual, penampilan hasil, dan penyimpanan citra hasil pengolahan. Ukuran kernel yang digunakan pada kedua metode adalah  $5 \times 5$ . Pemilihan kernel  $5 \times 5$  bertujuan untuk memberikan efek penghalusan yang terlihat tanpa menghilangkan terlalu banyak detail objek. Kernel tersebut menggunakan 25 nilai piksel yang berada di sekitar piksel pusat. Pada median filtering, nilai piksel baru ditentukan berdasarkan nilai median. Pada mean filtering, nilai piksel baru ditentukan berdasarkan nilai rata-rata.

#### 3.2 Perangkat yang digunakan

```
In [1]: !pip install opencv-python numpy matplotlib

Requirement already satisfied: opencv-python in c:\users\lenovo\documents\dean ganteng\lib\site-packages (4.13.0.92)
Requirement already satisfied: numpy in c:\users\lenovo\documents\dean ganteng\lib\site-packages (1.20.1)
Requirement already satisfied: matplotlib in c:\users\lenovo\documents\dean ganteng\lib\site-packages (3.3.4)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in c:\users\lenovo\documents\dean ganteng\lib\site-packages (from matplotlib) (2.4.7)
Requirement already satisfied: cycler>=0.10 in c:\users\lenovo\documents\dean ganteng\lib\site-packages (from matplotlib) (0.10.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\lenovo\documents\dean ganteng\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: python-dateutil>=2.1 in c:\users\lenovo\documents\dean ganteng\lib\site-packages (from matplotlib) (2.8.1)
Requirement already satisfied: pillow>=6.2.0 in c:\users\lenovo\documents\dean ganteng\lib\site-packages (from matplotlib) (8.2.0)
Requirement already satisfied: six in c:\users\lenovo\documents\dean ganteng\lib\site-packages (from cycler>=0.10->matplotlib) (1.15.0)

In [2]: import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

Perintah tersebut memastikan bahwa OpenCV, NumPy, dan Matplotlib telah tersedia pada lingkungan Jupyter Notebook. Setelah proses instalasi selesai, seluruh pustaka diimpor menggunakan kode berikut. Pemanggilan pustaka tersebut dilakukan pada awal program agar seluruh fungsi yang dibutuhkan dapat digunakan pada tahapan berikutnya.

### 3.3 Input Citra

```
In [3]: nama_file = "dean.jpg"

if not Path(nama_file).exists():
    raise FileNotFoundError(
        f"File '{nama_file}' tidak ditemukan. "
        "Pastikan foto berada dalam folder yang sama dengan notebook."
    )

citra_bgr = cv2.imread(nama_file)

if citra_bgr is None:
    raise ValueError("Gambar gagal dibaca. Periksa nama atau format file.")

# Konversi warna BGR ke RGB
citra_rgb = cv2.cvtColor(citra_bgr, cv2.COLOR_BGR2RGB)

print("Ukuran citra:", citra_bgr.shape)

Ukuran citra: (1280, 960, 3)
```

```
In [4]: plt.figure(figsize=(6, 8))
plt.imshow(citra_rgb)
plt.title("Citra Asli")
plt.axis("off")
plt.show()
```



Citra yang digunakan dalam proses filtering disimpan dengan nama dean.jpg. File citra ditempatkan pada folder yang sama dengan file Jupyter Notebook. Penempatan tersebut memudahkan program dalam menemukan dan membaca citra tanpa menggunakan alamat folder yang panjang. Program terlebih dahulu menyimpan nama file pada variabel nama\_file. Selanjutnya, program menggunakan Path(nama\_file).exists() untuk memeriksa keberadaan file. Apabila file tidak ditemukan, program menampilkan pesan kesalahan melalui FileNotFoundError. Pemeriksaan tersebut penting karena kesalahan nama file atau lokasi file dapat menyebabkan program gagal membaca citra. Setelah file ditemukan, program membaca citra menggunakan fungsi cv2.imread() dan menyimpannya dalam variabel citra\_bgr. OpenCV membaca citra menggunakan susunan kanal BGR, yaitu blue, green, dan red. Matplotlib menggunakan susunan kanal RGB, yaitu red, green, dan blue. Apabila citra BGR langsung ditampilkan menggunakan Matplotlib, warna pada gambar dapat terlihat tidak sesuai dengan citra asli. Oleh karena itu, program mengubah citra BGR menjadi RGB menggunakan fungsi berikut.

### 3.4 Penampakan Citra Asli

```
In [4]: plt.figure(figsize=(6, 8))
plt.imshow(citra_rgb)
plt.title("Citra Asli")
plt.axis("off")
plt.show()
```



Fungsi `plt.figure(figsize=(6, 8))` digunakan untuk menentukan ukuran tampilan gambar. Fungsi `plt.imshow(citra_rgb)` digunakan untuk menampilkan citra RGB. Judul gambar diberikan melalui fungsi `plt.title("Citra Asli")`. Perintah `plt.axis("off")` digunakan untuk menghilangkan garis sumbu dan angka koordinat di sekitar citra. Penghilangan sumbu membuat gambar terlihat lebih bersih dan memudahkan pengamatan objek. Fungsi `plt.show()` digunakan untuk menampilkan hasil pada Jupyter Notebook. Tahap ini belum mengubah nilai piksel citra. Program hanya menampilkan citra hasil pembacaan dan konversi warna.

### 3.5 Median Filtering

```
In [5]: ukuran_kernel_median = 5

hasil_median_bgr = cv2.medianBlur(
    citra_bgr,
    ukuran_kernel_median
)

hasil_median_rgb = cv2.cvtColor(
    hasil_median_bgr,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(6, 8))
plt.imshow(hasil_median_rgb)
plt.title("Hasil Median Filtering")
plt.axis("off")
plt.show()
```



Variabel ukuran\_kernel\_median diberi nilai 5. Nilai tersebut menunjukkan bahwa program menggunakan area berukuran  $5 \times 5$  piksel pada setiap proses filtering. Pada setiap posisi piksel, program mengambil 25 nilai piksel yang berada dalam area kernel. Nilai-nilai tersebut diurutkan dari nilai terkecil sampai nilai terbesar. Program kemudian mengambil nilai tengah sebagai nilai piksel baru.

Hasil fungsi cv2.medianBlur() masih menggunakan format BGR. Program mengubahnya menjadi RGB agar warna hasil filtering dapat ditampilkan dengan benar menggunakan Matplotlib. Median filtering menghasilkan citra yang lebih halus. Perubahan paling terlihat pada bagian tekstur kecil, permukaan tanah, pakaian, dan latar belakang. Meskipun citra mengalami penghalusan, bentuk utama objek masih dapat dikenali dengan jelas.

### 3.6 Konversi Grayscale

```
In [6]: citra_gray = cv2.cvtColor(
        citra_bgr,
        cv2.COLOR_BGR2GRAY
    )

    plt.figure(figsize=(6, 8))
    plt.imshow(citra_gray, cmap="gray")
    plt.title("Citra Grayscale")
    plt.axis("off")
    plt.show()
```



Sebelum melakukan mean filtering, program mengubah citra berwarna menjadi citra grayscale. Citra grayscale hanya mempunyai satu kanal intensitas. Setiap piksel mempunyai nilai antara 0 sampai 255. Nilai 0 menunjukkan warna hitam, nilai 255 menunjukkan warna putih, sedangkan nilai di antara 0 dan 255 menunjukkan berbagai tingkat keabuan. Fungsi cv2.cvtColor() digunakan untuk mengubah ruang warna citra. Parameter cv2.COLOR\_BGR2GRAY menunjukkan bahwa program mengubah citra BGR menjadi grayscale.

Konversi grayscale menyederhanakan proses mean filtering karena program hanya perlu menghitung satu nilai intensitas pada setiap piksel. Apabila mean filtering dilakukan langsung pada citra berwarna, program harus menghitung nilai rata-rata pada tiga kanal warna secara terpisah. Citra grayscale masih mempertahankan bentuk, tekstur, dan

perbedaan terang gelap pada objek. Namun, informasi warna pada langit, pakaian, tanah, dan tumbuhan telah dihilangkan.

### 3.7 Mean Filtering Manual

```
In [7]: def mean_filter_manual(citra, ukuran_kernel=5):

    # Memastikan ukuran kernel ganjil dan positif
    if ukuran_kernel < 1 or ukuran_kernel % 2 == 0:
        raise ValueError(
            "Ukuran kernel harus berupa bilangan ganjil positif."
        )

    # Menentukan padding
    padding = ukuran_kernel // 2

    # Menambahkan padding pada tepi citra
    citra_padding = np.pad(
        citra,
        pad_width=padding,
        mode="edge"
    )

    # Membuat array kosong untuk hasil filtering
    hasil = np.zeros_like(citra, dtype=np.uint8)

    # Mengambil tinggi dan lebar citra
    tinggi, lebar = citra.shape

    # Menghitung jumlah piksel dalam kernel
    jumlah_piksel_kernel = ukuran_kernel * ukuran_kernel

    # Memproses setiap piksel
    for baris in range(tinggi):
        for kolom in range(lebar):

            # Mengambil area sesuai ukuran kernel
            area_kernel = citra_padding[
                baris:baris + ukuran_kernel,
                kolom:kolom + ukuran_kernel
            ]

            # Menjumlahkan nilai piksel dalam area kernel
            jumlah_nilai = np.sum(
                area_kernel,
                dtype=np.uint32
            )

            # Menghitung rata-rata
            nilai_mean = jumlah_nilai / jumlah_piksel_kernel

            # Menyimpan hasil ke citra baru
            hasil[baris, kolom] = np.uint8(
                np.clip(round(nilai_mean), 0, 255)
            )

    return hasil
```

Mean filtering merupakan metode penghalusan citra yang mengganti nilai setiap piksel dengan nilai rata-rata piksel di sekitarnya. Pada tugas ini, proses mean filtering dibuat secara manual dan tidak menggunakan fungsi `cv2.blur()` atau fungsi mean filtering bawaan OpenCV. Mean filtering menggunakan kernel berukuran  $5 \times 5$ . Setiap kernel terdiri atas 25 nilai piksel. Program menjumlahkan seluruh nilai piksel tersebut, kemudian membaginya dengan 25.

### 3.8 Perbandingan Hasil Filtering

```
In [8]: ukuran_kernel_mean = 5

hasil_mean = mean_filter_manual(
    citra_gray,
    ukuran_kernel_mean
)

plt.figure(figsize=(6, 8))
plt.imshow(hasil_mean, cmap="gray")
plt.title("Hasil Mean Filtering Manual")
plt.axis("off")
plt.show()
```



Program menetapkan ukuran kernel sebesar 5. Citra grayscale dan nilai kernel tersebut dimasukkan ke dalam fungsi `mean_filter_manual()`. Hasil filtering disimpan dalam variabel `hasil_mean`. Selanjutnya, hasil ditampilkan menggunakan Matplotlib dengan pengaturan `cmap="gray"`. Mean filtering menghasilkan citra yang lebih halus dibandingkan citra grayscale asli. Namun, metode ini juga mengurangi ketajaman batas objek. Bagian wajah, pakaian, tanaman, tanah, dan pegunungan terlihat sedikit lebih kabur karena setiap piksel diganti dengan nilai rata-rata lingkungan sekitarnya.

### 3.9 Penyimpanan Hasil Citra

```
In [10]: cv2.imwrite(
    "hasil_median_filtering.jpg",
    hasil_median_bgr
)

cv2.imwrite(
    "citra_grayscale.jpg",
    citra_gray
)

cv2.imwrite(
    "hasil_mean_filtering_manual.jpg",
    hasil_mean
)

print("Semua hasil berhasil disimpan.")
Semua hasil berhasil disimpan.
```



```
In [9]: plt.figure(figsize=(12, 14))

plt.subplot(2, 2, 1)
plt.imshow(citra_rgb)
plt.title("Citra Asli")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(hasil_median_rgb)
plt.title("Hasil Median Filtering")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(citra_gray, cmap="gray")
plt.title("Original Image Grayscale")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(hasil_mean, cmap="gray")
plt.title("Mean Filtered image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Fungsi `plt.subplot (2, 2, nomor)` membagi tampilan menjadi dua baris dan dua kolom. Setiap posisi digunakan untuk menampilkan satu citra. Posisi pertama menampilkan citra asli. Posisi kedua menampilkan hasil median filtering. Posisi ketiga menampilkan citra grayscale. Posisi keempat menampilkan hasil mean filtering manual.

Perbandingan menunjukkan bahwa median filtering dan mean filtering menghasilkan efek yang berbeda. Median filtering menghaluskan citra dengan menggunakan nilai median. Metode ini masih mempertahankan tepi dan bentuk objek dengan cukup baik. Mean filtering menggunakan nilai rata-rata. Metode ini membuat transisi antar-piksel menjadi lebih lembut, tetapi juga menimbulkan efek blur yang lebih kuat. Detail kecil pada citra menjadi kurang tajam.



## BAB IV

### PENUTUP

#### 4.1 Kesimpulan

Berdasarkan landasan teori dan hasil praktikum pengolahan citra yang telah dilakukan, citra digital merupakan representasi visual dalam bentuk kumpulan piksel yang dapat diproses menggunakan komputer. Setiap piksel menyimpan nilai intensitas atau informasi warna tertentu. Pada citra grayscale, setiap piksel memiliki satu nilai intensitas dengan rentang 0 sampai 255. Pada citra berwarna, setiap piksel terdiri atas beberapa kanal warna, seperti red, green, dan blue pada model warna RGB. Kualitas citra digital dapat mengalami penurunan akibat noise, pencahayaan yang tidak sesuai, keterbatasan sensor, proses penyimpanan, kompresi, dan transmisi data. Gangguan tersebut dapat mengubah nilai piksel sehingga citra terlihat kasar, memiliki bintik, atau kehilangan sebagian detail. Filtering citra dapat digunakan untuk mengurangi gangguan tersebut dengan mengubah nilai suatu piksel berdasarkan nilai piksel di sekitarnya. Filtering dapat dilakukan pada domain spasial dan domain frekuensi. Praktikum ini menggunakan filtering pada domain spasial karena program melakukan pengolahan secara langsung terhadap nilai-nilai piksel. Kernel digeser pada seluruh bagian citra dan digunakan untuk menentukan nilai piksel baru. Metode ini mudah dipahami dan dapat diterapkan menggunakan operasi lokal pada setiap piksel.

Berdasarkan cara perhitungannya, filter dapat dikelompokkan menjadi filter linear dan filter nonlinier. Mean filter termasuk filter linear karena menghitung nilai piksel keluaran menggunakan nilai rata-rata piksel di dalam kernel. Median filter termasuk filter nonlinier karena menentukan nilai piksel keluaran melalui proses pengurutan nilai dan pemilihan nilai tengah. Perbedaan proses tersebut menyebabkan kedua filter menghasilkan karakteristik citra yang berbeda. Praktikum menggunakan foto diri berukuran  $1.280 \times 960$  piksel dengan tiga kanal warna. Program membaca citra melalui fungsi `cv2.imread()`. OpenCV membaca citra dalam format BGR sehingga program melakukan konversi menjadi RGB sebelum menampilkannya menggunakan Matplotlib. Konversi tersebut diperlukan agar susunan warna pada citra hasil tampilan sesuai dengan citra asli. Median filtering diterapkan pada citra berwarna menggunakan fungsi `cv2.medianBlur()` dari OpenCV. Program menggunakan kernel berukuran  $5 \times 5$  sehingga setiap proses mempertimbangkan 25 piksel dalam suatu area. Program menentukan nilai median dari piksel-piksel tersebut dan menggunakan nilai median sebagai nilai piksel baru. Hasil median filtering menunjukkan citra yang lebih halus dibandingkan citra asli. Metode ini masih mampu mempertahankan bentuk objek dan batas antarbagian citra dengan cukup baik.

Sebelum menjalankan mean filtering, program mengubah citra berwarna menjadi grayscale menggunakan fungsi `cv2.cvtColor()` dengan parameter

`cv2.COLOR_BGR2GRAY`. Konversi grayscale menyederhanakan proses perhitungan karena setiap piksel hanya memiliki satu nilai intensitas. Meskipun informasi warna hilang, citra grayscale tetap mempertahankan bentuk objek, pola tekstur, dan perbedaan intensitas terang dan gelap. Mean filtering dibuat melalui perhitungan manual tanpa menggunakan fungsi `cv2.blur()` atau fungsi mean filter bawaan OpenCV. Program menggunakan kernel  $5 \times 5$  dan menghitung rata-rata dari 25 nilai piksel di dalam kernel. Program kemudian menggunakan nilai rata-rata tersebut sebagai nilai piksel baru. Padding dengan mode edge diterapkan agar piksel pada bagian batas citra tetap dapat diproses dan ukuran citra keluaran tetap sama dengan citra masukan.

Hasil mean filtering manual menunjukkan bahwa citra menjadi lebih halus, tetapi bagian tepi dan detail objek terlihat lebih kabur. Kondisi tersebut muncul karena nilai piksel pusat bercampur dengan nilai piksel di sekitarnya melalui proses perhitungan rata-rata. Perbedaan intensitas yang tajam pada batas objek menjadi lebih kecil sehingga tepi objek kehilangan sebagian ketajamannya. Perbandingan hasil menunjukkan bahwa median filtering lebih mampu mempertahankan batas dan bentuk objek dibandingkan mean filtering. Median filtering tidak mudah terpengaruh oleh nilai piksel ekstrem karena menggunakan nilai tengah. Mean filtering lebih mudah terpengaruh oleh perbedaan nilai piksel karena menggunakan seluruh nilai dalam perhitungan rata-rata. Oleh karena itu, median filtering lebih sesuai ketika penghalusan citra perlu dilakukan dengan tetap mempertahankan tepi objek, sedangkan mean filtering lebih sesuai untuk menghasilkan penghalusan sederhana pada citra yang memiliki variasi intensitas ringan.

Program berhasil menampilkan citra asli, hasil median filtering, citra grayscale, dan hasil mean filtering manual dalam susunan dua baris dan dua kolom. Program juga berhasil menyimpan hasil pengolahan dalam file `hasil_median_filtering.jpg`, `citra_grayscale.jpg`, `hasil_mean_filtering_manual.jpg`, dan `perbandingan_filtering.png`. Secara keseluruhan, tujuan praktikum telah tercapai. Program berhasil menerapkan median filtering menggunakan OpenCV dan mean filtering melalui perhitungan manual. Hasil praktikum memperlihatkan bahwa setiap metode filtering memiliki cara kerja, kelebihan, dan keterbatasan yang berbeda. Pemilihan metode filtering harus mempertimbangkan jenis gangguan, kebutuhan penghalusan, tingkat ketajaman, dan detail citra yang perlu dipertahankan.

#### 4.2 Saran

1. Praktikum selanjutnya sebaiknya menggunakan citra yang telah diberi noise terkontrol, seperti salt-and-pepper noise dan Gaussian noise. Langkah ini dapat memperjelas kemampuan setiap metode filtering dalam mengurangi gangguan pada citra.
2. Pengujian sebaiknya menggunakan beberapa ukuran kernel, seperti  $3 \times 3$ ,  $5 \times 5$ , dan  $7 \times 7$ . Perbandingan tersebut dapat menunjukkan pengaruh ukuran kernel terhadap tingkat penghalusan, ketajaman detail, dan waktu komputasi.

3. Penilaian hasil filtering sebaiknya tidak hanya dilakukan secara visual. Pengujian dapat dilengkapi dengan metode kuantitatif seperti Mean Squared Error, Peak Signal-to-Noise Ratio, dan Structural Similarity Index Measure.
4. Proses mean filtering manual sebaiknya dikembangkan menggunakan operasi array NumPy atau teknik vektorisasi. Cara tersebut dapat mempercepat proses perhitungan pada citra beresolusi tinggi.
5. Praktikum berikutnya dapat membandingkan median filter dan mean filter dengan metode lain, seperti Gaussian filter, bilateral filter, dan filter penajaman. Perbandingan tersebut dapat membantu menentukan metode yang paling sesuai untuk setiap karakteristik citra.

**DAFTAR PUSTAKA**

1. Desai, B., Paliwal, M., & Nagwanshi, K. K. (2022). Study on image filtering: Techniques, algorithm and applications. arXiv.
2. Elad, M., Kwar, B., & Vaksman, G. (2023). Image denoising: The deep learning revolution and beyond. *Proceedings of the National Academy of Sciences*, 120(19), e2303880120.
3. Liu, X., Pedersen, M., & Wang, R. (2022). Survey of natural image enhancement techniques: Classification, evaluation, challenges, and perspectives. *Digital Signal Processing*, 127, 103547.
4. Rezaee, A. (2021). Partition fuzzy median filter for image restoration. *Fuzzy Information and Engineering*.
5. Satpathy, S. K., Panda, S., Nagwanshi, K. K., & Ardil, C. (2022). Image restoration in non-linear filtering domain using MDB approach. arXiv.
6. Su, J., Xu, B., & Yin, H. (2022). A survey of deep learning approaches to image restoration. *Neurocomputing*, 487, 46–65.
7. OpenCV. (2025). Image filtering: Smoothing images. Open Source Computer Vision Library Documentation.
8. NumPy Developers. (2025). NumPy reference documentation. NumPy.
9. Matplotlib Development Team. (2025). Matplotlib documentation. Matplotlib.
10. Python Software Foundation. (2025). Python 3 documentation. Python Software Foundation.